

군집분석을 활용한

대출 고객 세분화 및 맞춤형 대출 상품 전략 제안

13기 이서연 이유정

14기 고성수 설수현 이하린 정아진

목차

1 분석 개요

2 데이터 전처리

3 EDA 및 최적 변수 선택

4 군집 알고리즘 성능 비교

5 실증 분석 및 군집별 비즈니스 전략

6 한계점 및 향후 보완

분석 개요

파일명	Loan_approval_data_2025.csv
관측치	50,000개
결측치	없음



1. 군집 세분화

- 대출 신청 고객 데이터를 기반으로 고객을 유사한 신용·재무 특성을 가진 군집으로 세분화



2. 페르소나 정의

- 각 군집을 고객 페르소나로 정의



3. 타겟팅 전략 제안

- 군집별 리스크 수준에 맞는 대출 상품 타겟팅 전략을 제안

데이터 전처리

상환능력

- ✓ `annual_income`
연간 총수입
- ✓ `savings_assets`
보유 중인 예금 및 자산 규모
- ✓ `years_employed`
현재 직장에서의 근무한 연수

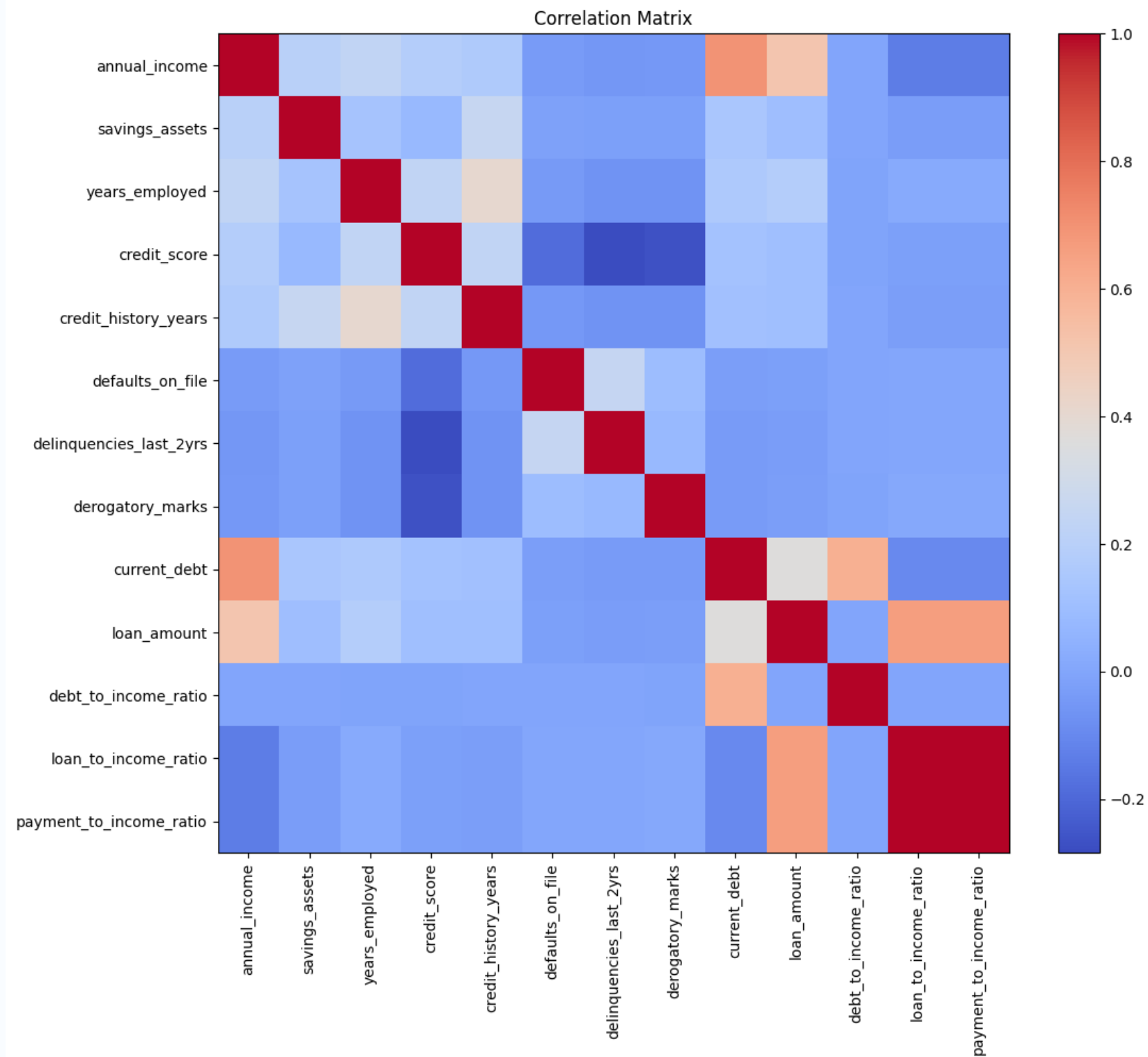
기존 신용상태

- ✓ `credit_score`
신용점수
- ✓ `credit_history_years`
신용 거래 기록이 유지된 연수
- ✓ `delinquencies_last_2yrs`
최근 2년간 연체 횟수
- ✓ `defaults_on_file`
과거에 채무 불이행이 있는지 여부
- ✓ `derogatory_marks`
신용 보고서상의 부정적인 기록 횟수

현재 대출부담

- ✓ `current_debt`
현재 가지고 있는 총 부채액
- ✓ `loan_amount`
신청한 대출 금액
- ✓ `debt_to_income_ratio (DTI)`
총 부채 상환 비율
- ✓ `loan_to_income_ratio`
소득 대비 대출 신청 금액 비율
- ✓ `payment_to_income_ratio`
월 상환액 대비 소득 비율

EDA



조합 1 - 전체 변수 조합



조합 2 - PTI 제거 조합



조합3 - LTI 제거 조합



조합 4 - 비율 변수 중심 조합



조합 5 - 핵심 축 축약 조합

최적 변수 선택

```
# 조합별 K-means 비교 함수 만들기 각 조합마다 자동 스케일링 k=2~5 실루엣 계수 계산
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import pandas as pd

def run_kmeans_compare(df, var_sets, k_range=range(2, 6)):
    results = []

    for set_name, vars_list in var_sets.items():
        print(f"\n현재 변수 조합: {set_name}")

        X = df[vars_list].copy()

        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        for k in k_range:
            print(f" k={k} 계산 중...")

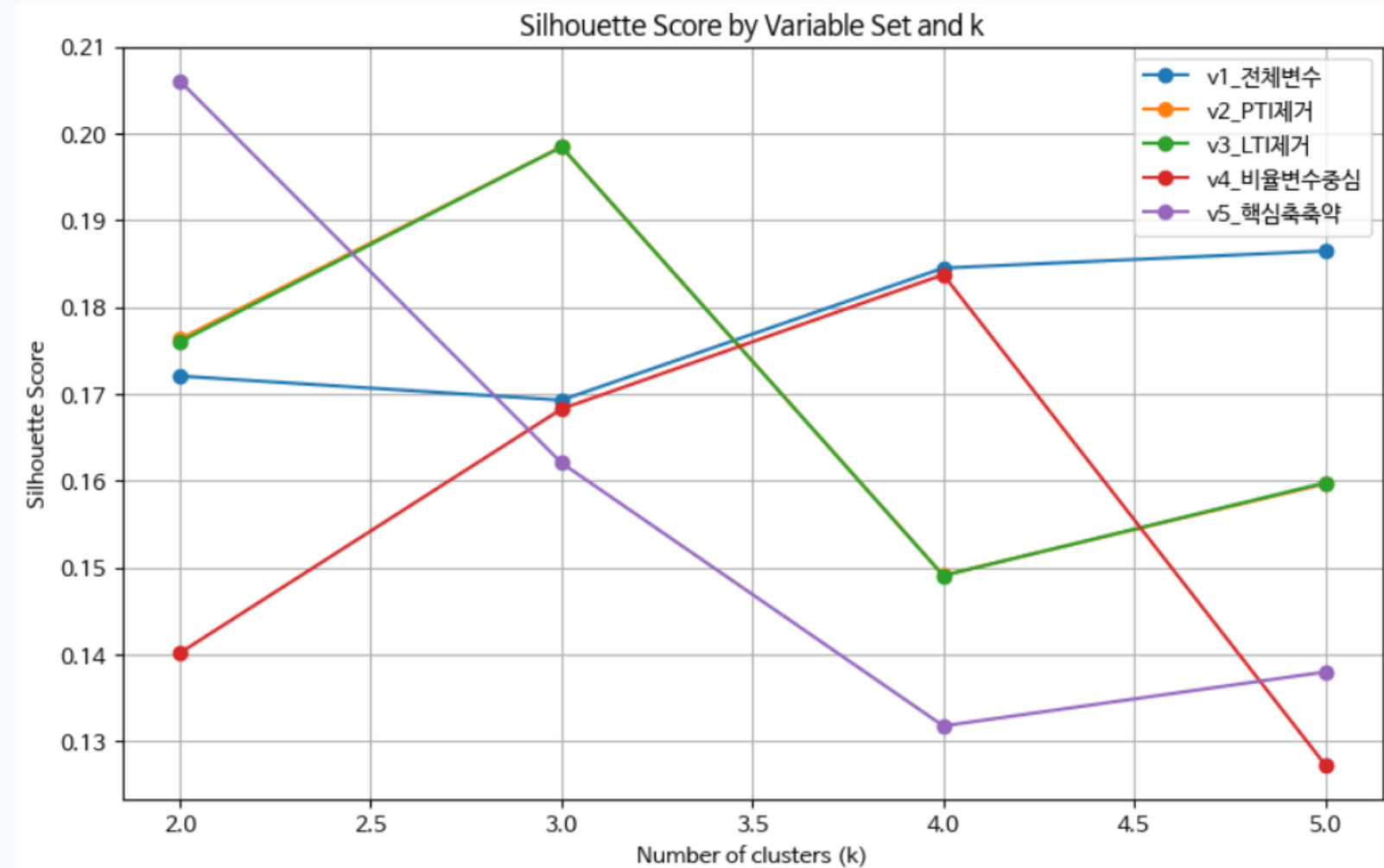
            kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
            labels = kmeans.fit_predict(X_scaled)

            score = silhouette_score(
                X_scaled,
                labels,
                sample_size=10000,
                random_state=42
            )

            results.append({
                '변수조합': set_name,
                '변수개수': len(vars_list),
                'k': k,
                'silhouette_score': score
            })

    return pd.DataFrame(results)
```

	변수조합	변수개수	k	silhouette_score
16	v5_핵심축축약	9	2	0.206077
9	v3_LTI제거	12	3	0.198470
5	v2_PTI제거	12	3	0.198469
3	v1_전체변수	13	5	0.186487
14	v4_비율변수중심	10	4	0.183753

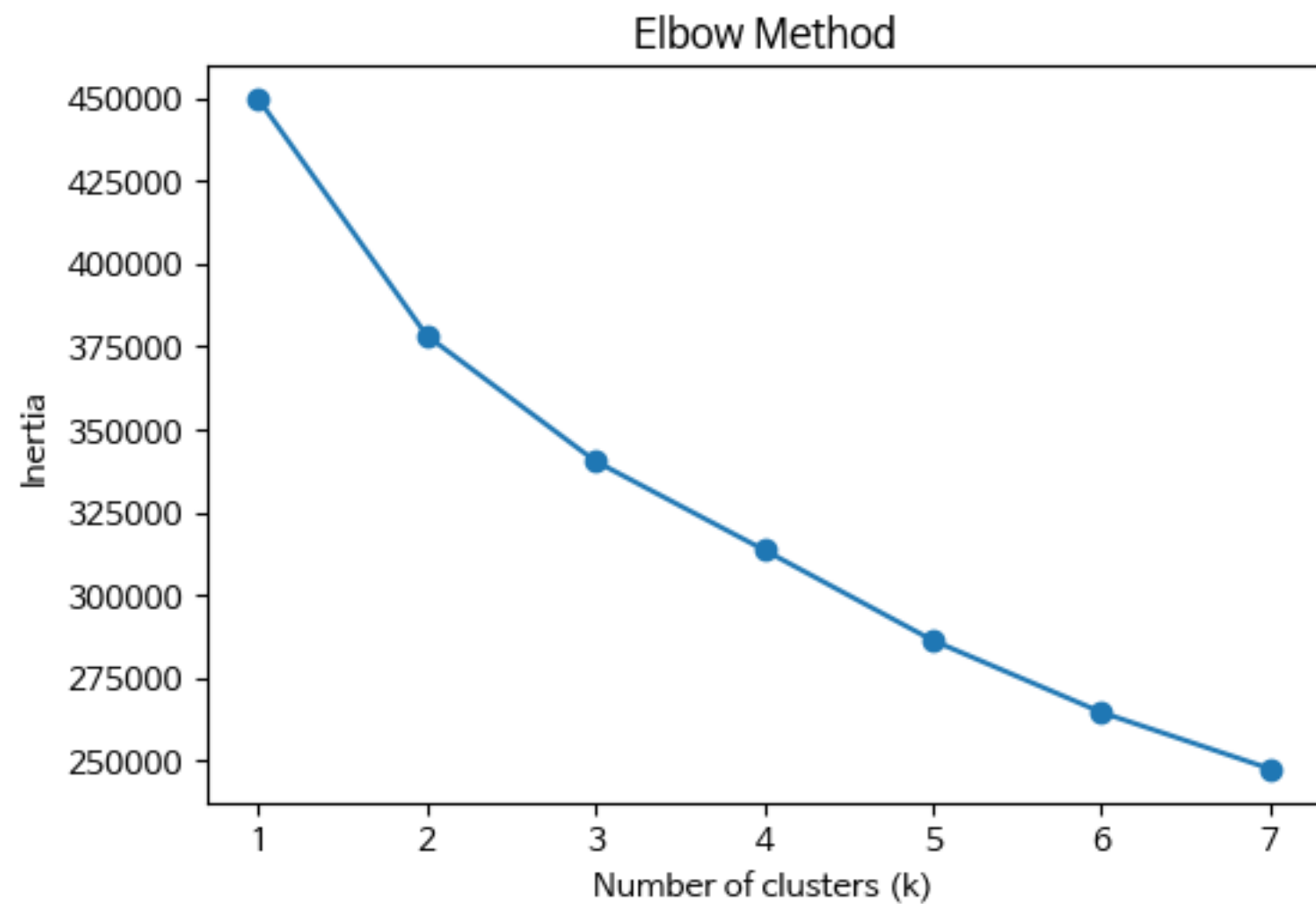


K-means 군집화 결과 (k=2)

Cluster 0 저위험 안정형 고객군	
16,246명 (32.5%)	
연소득 / 저축성 자산	높음 ↑
신용점수 / 근속연수	높음 ↑
신용거래 기간	신용 이력 풍부 ↑
최근 2년 연체 횟수	낮음 ↓
LTI (소득 대비 대출)	낮음 ↓
승인율	69.76%

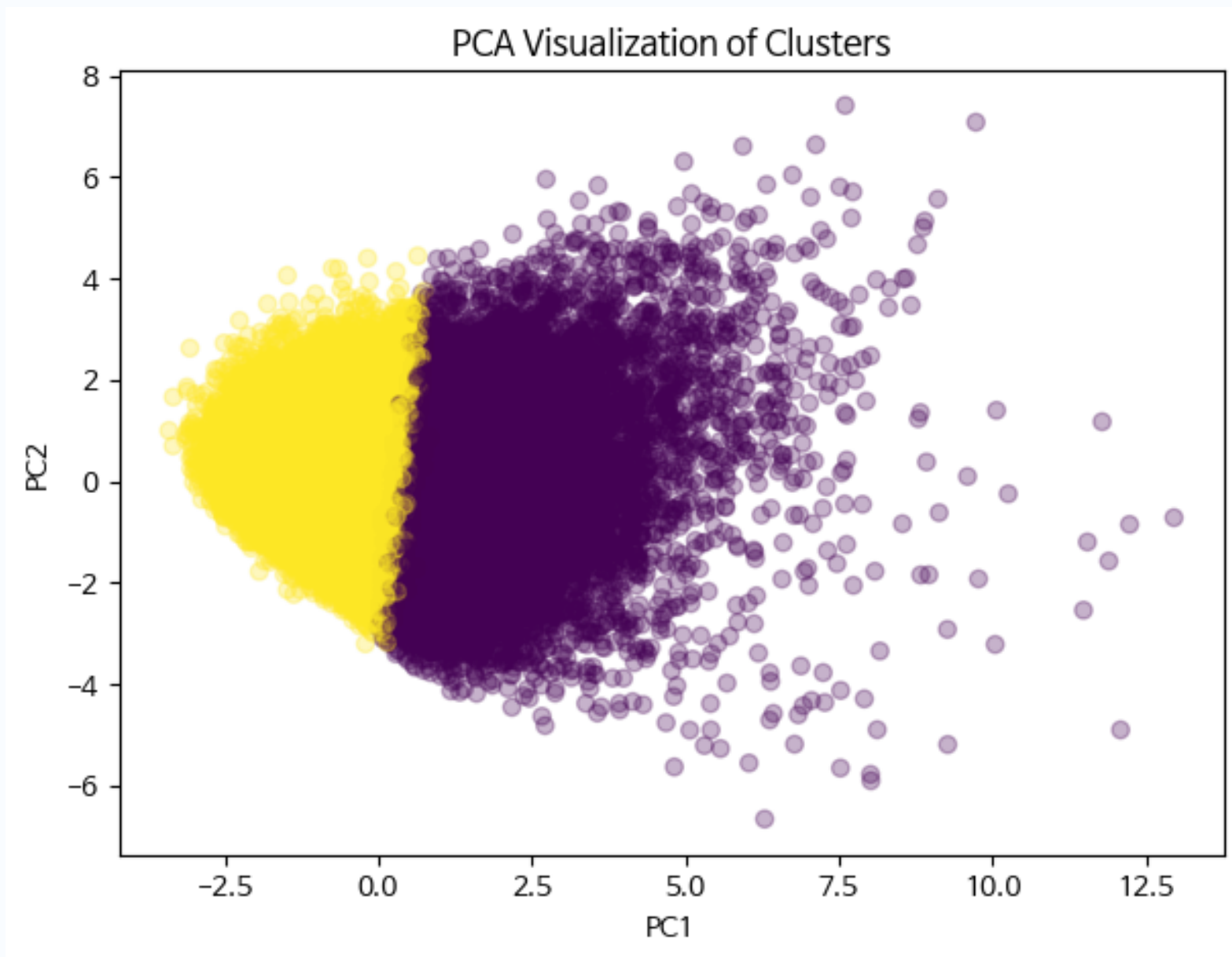
Cluster 1 잠재위험 상환부담형	
33,754명 (67.5%)	
연소득 / 저축성 자산	낮음 ↓
신용점수 / 근속연수	낮음 ↓
신용거래 기간	짧음 ↓
최근 2년 연체 횟수	높음 ↑
LTI (소득 대비 대출)	높음 ↑
승인율	47.96%

군집 모델 타당성 검증



	변수	Cluster0_mean	Cluster1_mean	t_statistic	p_value
0	annual_income	76556.0150	37311.5952	122.3371	0.0
1	savings_assets	8267.7924	1346.8749	39.9434	0.0
2	years_employed	13.1425	4.7174	113.7537	0.0
3	credit_score	677.6570	627.2301	88.1462	0.0
4	credit_history_years	13.1772	5.7575	108.7845	0.0
5	delinquencies_last_2yrs	0.3703	0.6434	-37.7109	0.0
6	current_debt	25169.0814	9054.4888	117.9694	0.0
7	debt_to_income_ratio	0.3438	0.2578	56.0675	0.0
8	loan_to_income_ratio	0.6355	0.7340	-23.6739	0.0

PCA 기반 군집 시각화 및 주성분 기여도 분석



PCA 결과

- PC1 (설명력 25.4%) : 부채 규모, 연소득, 근속 연수 등 고객의 전반적인 금융 규모를 나타냅니다.
- PC2 (설명력 16.9%) : DTI, 최근 연체 횟수 등 부채 부담 및 신용 위험도를 나타냅니다.
- 두 주성분을 통해 군집이 PC1 방향을 중심으로 선명하게 분리됨을 확인하였습니다.

군집 알고리즘 성능 비교 (GMM)

```
1 # GMM
2 from sklearn.mixture import GaussianMixture
3 from sklearn.metrics import silhouette_score
4 import pandas as pd
5 import numpy as np
6
7 gmm_results = []
8
9 for k in range(2, 6):
10     gmm = GaussianMixture(n_components=k, random_state=42)
11     labels = gmm.fit_predict(X_final_scaled)
12
13     score = silhouette_score(
14         X_final_scaled,
15         labels,
16         sample_size=10000,
17         random_state=42
18     )
19
20     gmm_results.append({
21         'model': 'GMM',
22         'k': k,
23         'n_clusters': len(set(labels)),
24         'silhouette_score': score
25     })
26
27 gmm_results = pd.DataFrame(gmm_results)
28 gmm_results
```

model	k	n_clusters	silhouette_score
GMM	2	2	0.162424
GMM	3	3	0.043480
GMM	4	4	0.038339
GMM	5	5	0.016726

- 금융 데이터의 선형적 특징을 K-Means보다 선명하게 포착하지 못함
- K-Means (0.206) 대비 GMM (0.162)의 상대적 성능 저하

군집 알고리즘 성능 비교(계층적 군집화)

```
1 # 계층적 군집화 표본 10000개
2 from sklearn.cluster import AgglomerativeClustering
3
4 # 계층적 군집화는 계산량이 커서 표본으로 비교
5 sample_idx = df.sample(n=10000, random_state=42).index
6 X_sample = X_final_scaled[sample_idx]
7
8 hierarchical_results = []
9
10 for k in range(2, 6):
11     hc = AgglomerativeClustering(n_clusters=k, linkage='ward')
12     labels = hc.fit_predict(X_sample)
13
14     score = silhouette_score(X_sample, labels)
15
16     hierarchical_results.append({
17         'model': 'Hierarchical',
18         'k': k,
19         'n_clusters': len(set(labels)),
20         'silhouette_score': score
21     })
22
23 hierarchical_results = pd.DataFrame(hierarchical_results)
24 hierarchical_results
```

	model	k	n_clusters	silhouette_score
0	Hierarchical	2	2	0.193936
1	Hierarchical	3	3	0.189069
2	Hierarchical	4	4	0.068109
3	Hierarchical	5	5	0.079624

- k=2에서 실루엣계수 0.1939로 비교적 양호한 군집 성능 확인
- K-means k=2 (0.2061)보다 낮고, 대규모 데이터 적용 시 계산 비용이 큼
- 차선 후보로 검토하되, 최종 모델은 K-means k=2로 선정

군집 알고리즘 성능 비교 (DBSCAN)

```
1 # DBSCAN은 k가 아니라 eps, min_samples를 조정
2 from sklearn.cluster import DBSCAN
3
4 # DBSCAN도 계산량이 커서 표본으로 비교
5 sample_idx = df.sample(n=10000, random_state=42).index
6 X_sample = X_final_scaled[sample_idx]
7
8 dbscan_results = []
9
10 eps_list = [0.8, 1.0, 1.2, 1.5]
11 min_samples_list = [5, 10]
12
13 for eps in eps_list:
14     for min_samples in min_samples_list:
15         dbscan = DBSCAN(eps=eps, min_samples=min_samples)
16         labels = dbscan.fit_predict(X_sample)
17
18         n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
19         n_noise = list(labels).count(-1)
20
21         # 군집이 2개 이상일 때만 실루엣 계산
22         if n_clusters >= 2:
23             score = silhouette_score(X_sample, labels)
24         else:
25             score = np.nan
26
27         dbscan_results.append({
28             'model': 'DBSCAN',
29             'eps': eps,
30             'min_samples': min_samples,
31             'n_clusters': n_clusters,
32             'n_noise': n_noise,
33             'silhouette_score': score
34         })
35
36 dbscan_results = pd.DataFrame(dbscan_results)
37 dbscan_results
```

	model	eps	min_samples	n_clusters	n_noise	silhouette_score
0	DBSCAN	0.8	5	40	4821	-0.340193
1	DBSCAN	0.8	10	10	6114	-0.279371
2	DBSCAN	1.0	5	23	2664	-0.215749
3	DBSCAN	1.0	10	8	3612	-0.136895
4	DBSCAN	1.2	5	9	1444	-0.026025
5	DBSCAN	1.2	10	2	1989	0.045264
6	DBSCAN	1.5	5	5	579	0.098626
7	DBSCAN	1.5	10	1	763	NaN

- 표본 10,000건 기준, eps=0.8, min_samples=5에서 약 40% 이상이 노이즈로 분류되고, 실루엣계수도 -0.3402로 낮게 나타남
- 최고 실루엣계수도 0.0986으로 K-means k=2 (0.2061)보다 낮음
- 설정값에 민감하여 안정적인 군집 모델로 활용하기 어려움

군집 알고리즘 성능 비교 (Meanshift)

```
1 # Mean Shift 자동으로 군집 수 정해짐
2 from sklearn.cluster import MeanShift, estimate_bandwidth
3
4 # Mean Shift도 표본으로 비교
5 sample_idx = df.sample(n=10000, random_state=42).index
6 X_sample = X_final_scaled[sample_idx]
7
8 bandwidth = estimate_bandwidth(X_sample, quantile=0.2, n_samples=3000, random_state=42)
9
10 meanshift = MeanShift(bandwidth=bandwidth, bin_seeding=True)
11 ms_labels = meanshift.fit_predict(X_sample)
12
13 n_clusters = len(set(ms_labels))
14
15 if n_clusters >= 2:
16     ms_score = silhouette_score(X_sample, ms_labels)
17 else:
18     ms_score = np.nan
19
20 meanshift_results = pd.DataFrame([
21     {'model': 'MeanShift',
22      'bandwidth': bandwidth,
23      'n_clusters': n_clusters,
24      'silhouette_score': ms_score
25 }])
26
27 meanshift_results
```

- 실루엣 계수 0.3833으로 매우 높음
- 18개 군집으로 세분화되어 전략 해석이 어려움
- 최종 모델이 아닌 비교 모델로 활용

	model	bandwidth	n_clusters	silhouette_score
0	MeanShift	3.01319	18	0.383289

최종 모델 선정 결과

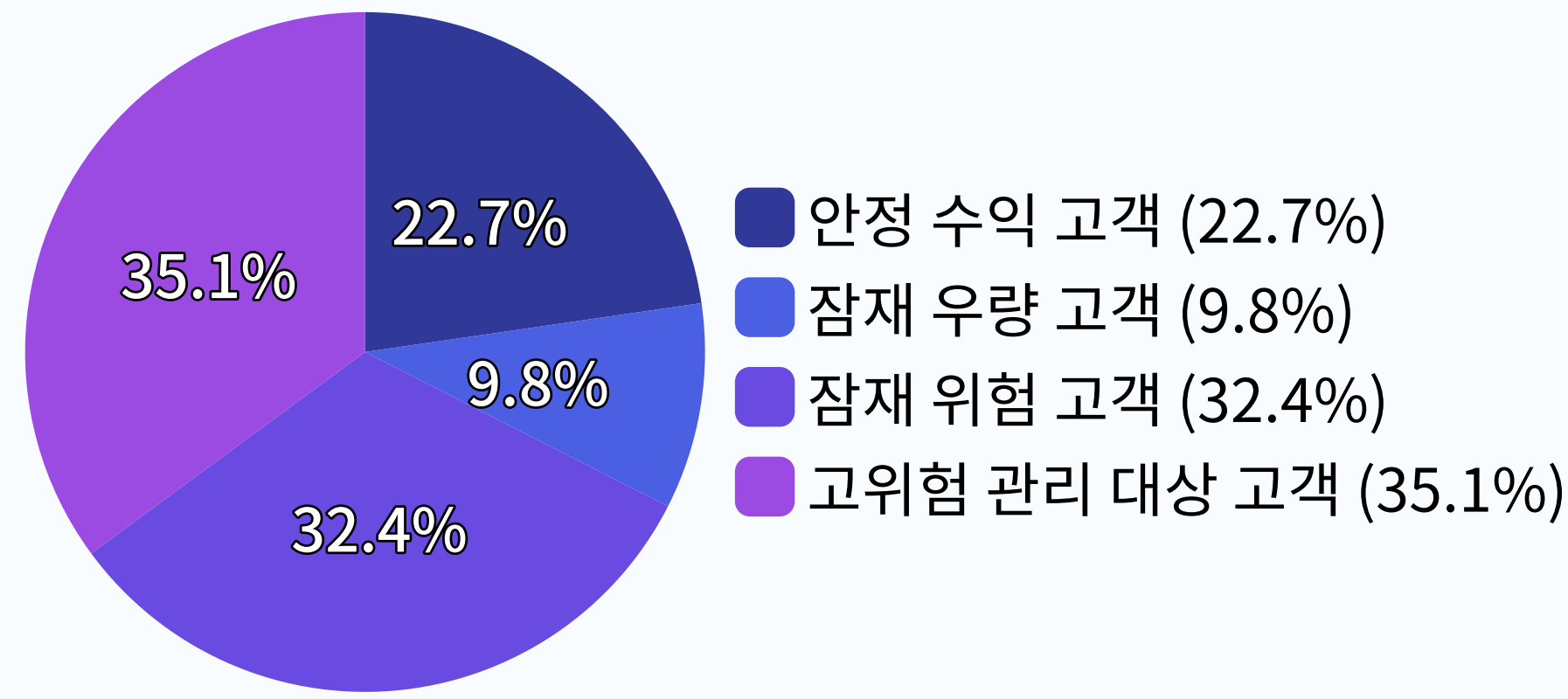
모델	최고 조건	군집 수	최고 실루엣 계수	판단
K-means	k = 2	2개	0.2061	최종 모델
계층적 군집화	k = 2	2개	0.1939	차선 후보
GMM	k = 2	2개	0.1624	성능 낮음
DBSCAN	eps=1.5, min_samples=5	5개	0.0986	노이즈 많음
Mean Shift	bandwidth = 3.013	18개	0.3833	해석 복잡

실루엣계수만이 아니라 해석 가능성, 승인율 검증, 전략 연결성 종합하여 K-means k=2를 최종 모델로 선정

대출 고객 4개 타겟 도출

```
1 def assign_target(row):
2     if row['cluster'] == 0 and row['loan_status'] == 1:
3         return '안정 수익 고객'
4     elif row['cluster'] == 0 and row['loan_status'] == 0:
5         return '잠재 우량 고객'
6     elif row['cluster'] == 1 and row['loan_status'] == 1:
7         return '잠재 위험 고객'
8     elif row['cluster'] == 1 and row['loan_status'] == 0:
9         return '고위험 관리 대상 고객'
10
11 df['target_segment'] = df.apply(assign_target, axis=1)
12
13 target_counts = df['target_segment'].value_counts()
14 target_ratio = df['target_segment'].value_counts(normalize=True) * 100
15
16 target_summary = pd.DataFrame({
17     '고객 수': target_counts,
18     '비율(%)': target_ratio.round(2)
19 })
20
21 target_summary
```

	고객 수	비율(%)
target_segment		
고위험 관리 대상 고객	17564	35.13
잠재 위험 고객	16190	32.38
안정 수익 고객	11333	22.67
잠재 우량 고객	4913	9.83



Cluster 0/1의 신용·재무 특성에 실제 승인 여부를 결합하여 은행의 수익성과 리스크 관리 관점에서 4개 타겟 도출

타겟별 금융 전략 제안

안정 수익 고객 (Cluster 0, 승인)

- 11,333명 (22.7%)
- 고소득·고신용·긴 근속연수
- 전략 : 간편 대출로 우량 고객을 빠르게 선점하고 장기 고객화

잠재 우량 고객 (Cluster 0, 거절)

- 4,913명 (9.8%)
- 소득·근속은 양호하지만 부채와 DTI 높음
- 전략 : 기존 부채 부담을 낮춰 신규 수익 고객으로 전환

잠재 위험 고객 (Cluster 1, 승인)

- 16,190명 (32.4%)
- 소득·자산은 낮지만 현재 부채 부담은 낮음
- 전략 : 제한적 한도 내에서 관리형 수익 고객으로 육성

고위험 관리 대상 고객 (Cluster 1, 거절)

- 17,564명 (35.1%)
- 저신용·고연체·높은 LTI
- 전략 : 손실 방어와 신용회복 경로 안내

사후 해석 (연령 및 금리 차이)

	age	interest_rate
cluster		
0	42.97	14.41
1	31.10	16.02

- Cluster 0은 평균 42.97세로 비교적 연령대가 높으며, 평균 14.41%의 낮은 금리 혜택을 받고 있음
- Cluster 1은 평균 31세로 비교적 연령대가 낮으며, 평균 금리는 16.02%로 더 높음

사후 해석 (직업)

occupation_status	Employed	Self-Employed	Student
cluster			
0	70.28	28.21	1.51
1	69.78	16.58	13.64

- Cluster 0은 Self-Employed(자영업자) 비율이 28.21%로 Cluster 1보다 높고, Student(학생) 비율은 1.51%로 매우 낮음
- Cluster 1은 Student(학생) 비율이 13.64%로 Cluster 0보다 훨씬 높음

사후 해석 (대출 상품 유형)

product_type	Credit Card	Line of Credit	Personal Loan
cluster			
0	45.14	20.47	34.40
1	44.80	19.84	35.36

- Credit Card(신용카드) 비율
Cluster 0 : 45.14% / Cluster 1 : 44.80%
- Line of Credit(신용한도 대출) 비율
Cluster 0 : 20.47% / Cluster 1 : 19.84%
- Personal Loan(개인대출) 비율
Cluster 0 : 34.40%, Cluster 1 : 35.36%

사후 해석 (대출 목적)

loan_intent	Business	Debt Consolidation	Education	Home Improvement	Medical	Personal
cluster						
0	14.85	9.57	20.61	14.59	15.41	24.98
1	14.98	9.96	20.10	15.06	15.09	24.80

- Cluster 0과 Cluster 1 모두 개인 목적 비율이 약 25%로 가장 높았고, Education(교육 목적)이 약 20% 수준으로 그다음 높은 비율을 보였다.
- Business(사업 목적), Home Improvement(주택 개선), Medical(의료 목적), Debt Consolidation(부채 통합) 비율도 두 군집에서 유사하게 나타났다.

한계점 및 향후 보완

정적 데이터 기반 분석

- 한계점 : 현재 분석은 특정 시점의 스냅샷 데이터로, 고객의 신용 상태 변화를 반영하지 못함
- 보완점 : 시계열 데이터가 확보되면 동적 군집화를 적용하고, 일정 주기로 군집을 재학습하는 모니터링 파이프라인 구축

외부 정보 미반영

- 한계점 : 거시경제 지표, 지역 정보, 대출 목적의 세부 맥락 등 외부 요인이 충분히 반영되지 않음
- 보완점 : 외부 데이터(경기 지수, 지역 소득 수준 등)를 추가하여 군집 해석과 전략 제안의 정교화 가능

군집 규모 차이

- 한계점 : Cluster 0 (32.5%)과 Cluster 1 (67.5%) 간 규모 차이가 존재함
- 보완점 : 군집 규모 차이를 고려한 가중 샘플링 또는 재군집화 실험

모델 해석 가능성 vs 성능 트레이드오프

- 한계점 : Mean Shift는 실루엣 계수가 높았지만, 18개 군집으로 나뉘어 전략 해석이 복잡함
- 보완점 : Mean Shift 결과를 계층적으로 병합하여 4~6개 군집으로 축소하는 실험